

Lab3

Course: ARTI 404 – Image Processing

Lab Topic: Image Manipulation using the OpenCV Library

Student Name: Muhannad Almutlaq

Student ID: 2240006060

1. Lab Outcomes

The objective of this lab is to manipulate images with the OpenCV library and NumPy: reading, displaying, and saving images, converting color spaces, applying geometric transformations, and applying intensity (point-processing) transformations.

2. Procedural Tasks

Task 1: Reading and Displaying an Image

- **Description:** Imported OpenCV and loaded the image with `cv2.imread`, which returns a NumPy array in BGR order (here 480x640 with three channels), checked that the read succeeded, and displayed it in a window with `cv2.imshow`. `cv2.waitKey(0)` pauses until a key is pressed, `cv2.destroyAllWindows` closes the window, and the extra `cv2.waitKey(1)` lets the window close properly in the notebook.
- **Code & Output:**

```
import cv2
import sys

input_path= "./images/image1.png"
output_path1= "./images/image1_output1.png"
output_path2= "./images/image1_output2.png"

print("---- Step 1: Reading and displaying the original image ----")
image= cv2.imread(input_path)

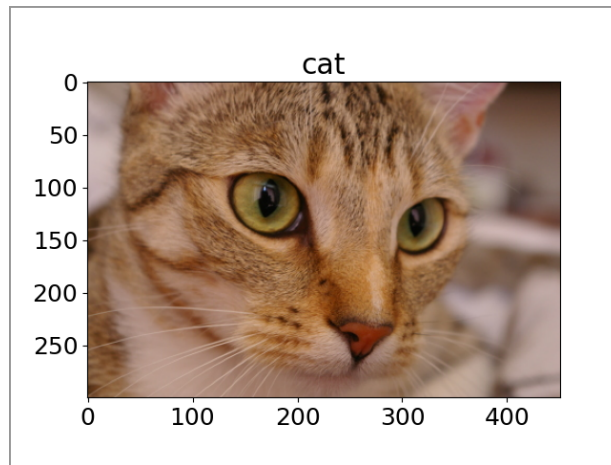
if image is None:
    print(f"Error: Could not read image at {input_path}. Please check the path.")
    sys.exit()

print(f"Data type of loaded image: {type(image)}")
```

```

cv2.imshow("Input image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.waitKey(1)
---- Step 1: Reading and displaying the original image ----
Data type of loaded image: <class 'numpy.ndarray'>
-1

```



Window "Input image"

Task 2: Loading an Image in Grayscale and Saving It

- **Description:** Loaded the same image directly as a grayscale image with `cv2.IMREAD_GRAYSCALE`, displayed it, and saved it to disk with `cv2.imwrite`.
- **Code & Output:**

```

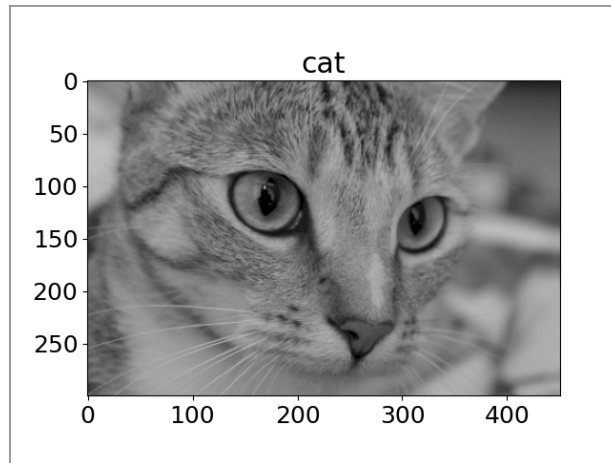
print("\n---- Step 2 & 3: Loading in grayscale and saving ----")
gray_image_direct= cv2.imread(input_path, cv2.IMREAD_GRAYSCALE)

cv2.imshow("Grayscale (Loaded Directly)", gray_image_direct)
cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.waitKey(1)

# Save the grayscale image
cv2.imwrite(output_path1, gray_image_direct)
print(f"Successfully saved grayscale image to: {output_path1}")

---- Step 2 & 3: Loading in grayscale and saving ----
Successfully saved grayscale image to: ./images/image1_output1.png

```



Window "Grayscale (Loaded Directly)"

Task 3: Converting to Grayscale with cvtColor

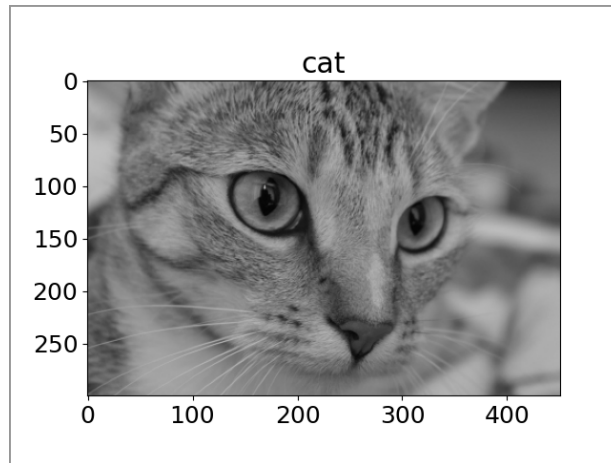
- **Description:** Converted the color image to grayscale with `cv2.cvtColor` and `cv2.COLOR_BGR2GRAY`, displayed it, and wrote a grayscale image to a second output file with `cv2.imwrite`. The result looks the same as the directly loaded grayscale image, but the pixel values differ slightly (on average by about 1.3 gray levels).
- **Code & Output:**

```
print("\n---- Step 4: Color space conversions ----")
gray_image_converted= cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

cv2.imshow("Grayscale (Converted using cvtColor)", gray_image_converted)
cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.waitKey(1)

# Save the inverted grayscale image
cv2.imwrite(output_path2, gray_image_direct)
print(f"Successfully saved inverted grayscale image to: {output_path2}")

---- Step 4: Color space conversions ----
Successfully saved inverted grayscale image to: ./images/image1_output2.png
```



Window "Grayscale (Converted using cvtColor)"

Task 4: Color Spaces and the YUV Channels

- Description:** Listed the color conversion flags available in OpenCV (374 in this version), converted the image to YUV with `cv2.COLOR_BGR2YUV`, and displayed the Y, U, and V channels separately using NumPy slicing. Y holds the brightness and looks like a grayscale version of the image. U (blue projection) and V (red projection) hold the color information, and in this image they span only narrow ranges (about 88 to 150 and 112 to 189), so they appear as low-contrast gray.
- Code & Output:**

```
color_flags= [x for x in dir(cv2) if x.startswith("COLOR_")]
print(f"Total color conversion options available: {len(color_flags)}")
print(f"First 10 flags as an example: {color_flags[:10]}")

yuv_image= cv2.cvtColor(image, cv2.COLOR_BGR2YUV)

# Display individual Y, U, and V channels
# Y= physical linear-space brightness (channel 0)
# U= blue projection (channel 1)
# V= red projection (channel 2)
cv2.imshow("Y channel (Brightness)", yuv_image[:, :, 0])
cv2.imshow("U channel (Blue Projection)", yuv_image[:, :, 1])
cv2.imshow("V channel (Red Projection)", yuv_image[:, :, 2])
cv2.waitKey(0)
cv2.destroyAllWindows()
Total color conversion options available: 374
First 10 flags as an example: ['COLOR_BAYER_BG2BGR', 'COLOR_BAYER_BG2BGRA',
'COLOR_BAYER_BG2BGR_EA', 'COLOR_BAYER_BG2BGR_VNG', 'COLOR_BAYER_BG2GRAY',
'COLOR_BAYER_BG2RGB', 'COLOR_BAYER_BG2RGBA', 'COLOR_BAYER_BG2RGB_EA',
'COLOR_BAYER_BG2RGB_VNG', 'COLOR_BAYER_BGGR2BGR']
```



Task 5: Geometric Transformations

- **Description:** Applied three geometric transformations to the image: scaling by 1.5 with cubic interpolation, rotation by 120 degrees around the center, and a shear along the X axis with a shear factor of 0.5. The windows below are drawn at the same scale as the 640-pixel-wide original.

- **Code & Output:**

5.1 Scaling (enlarging) the image

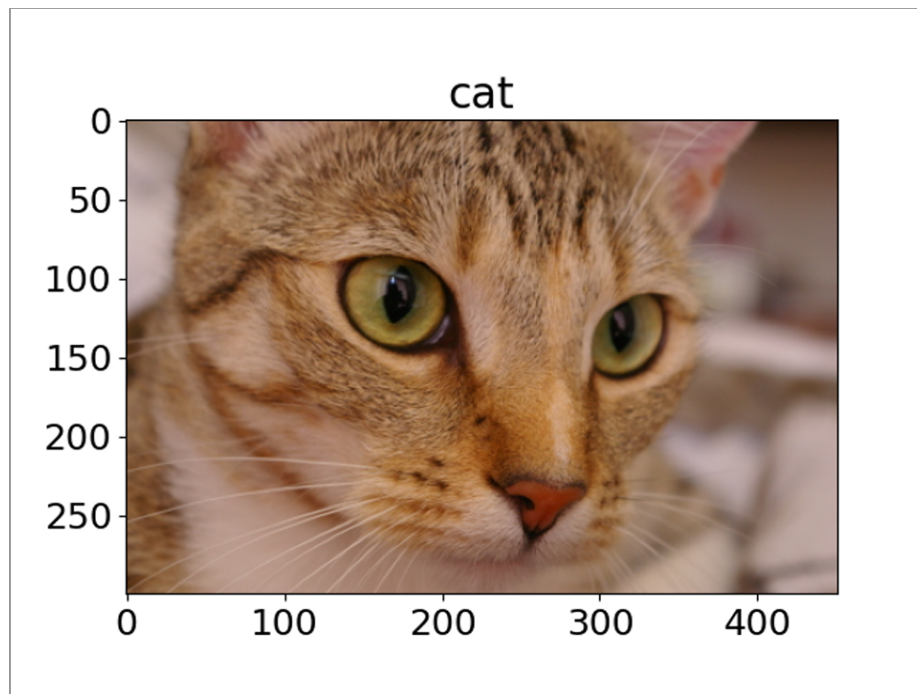
```
import numpy as np

input_path = "./images/image1.png"
image= cv2.imread(input_path)

if image is None:
    print(f"Error: Could not read image at {input_path}.")
    sys.exit()

(h, w)= image.shape[:2]

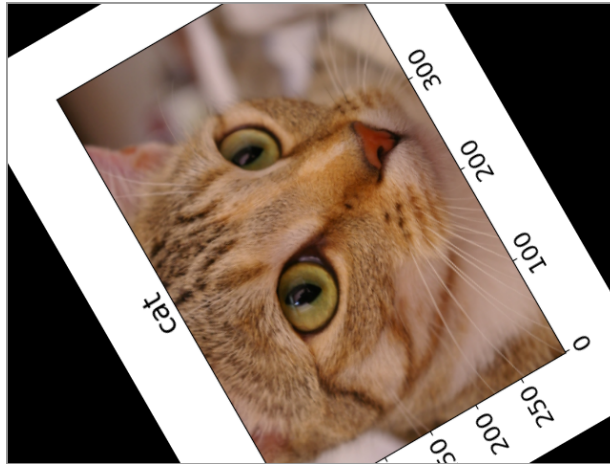
# 1. Increase the size of the image (Scaling up by 1.5x)
enlarged_image= cv2.resize(image, None, fx= 1.5, fy= 1.5, interpolation=
cv2.INTER_CUBIC)
cv2.imshow("1. Enlarged Image", enlarged_image)
```



cv2.resize with $fx = fy = 1.5$ and cubic interpolation enlarges the image from 480x640 to 720x960.

5.2 Rotating the image by 120 degrees

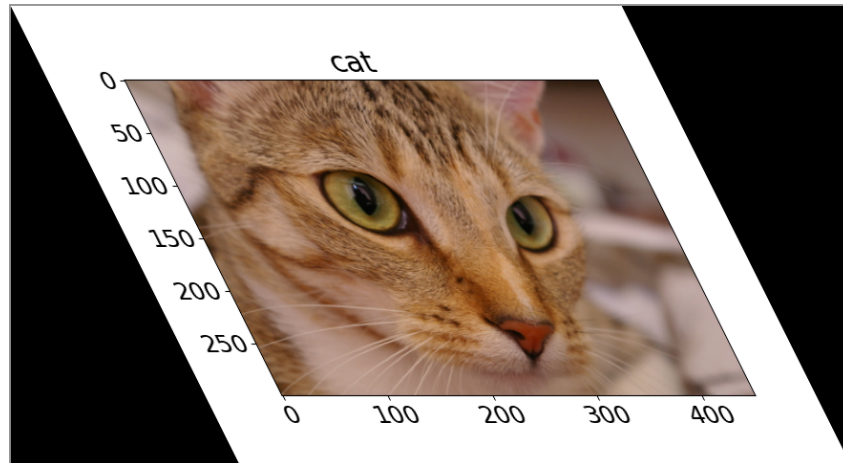
```
# 2. Rotate the image by 120 degrees
center= (w // 2, h // 2)
M_rot= cv2.getRotationMatrix2D(center, 120, 1.0)
rotated_img= cv2.warpAffine(image, M_rot, (w, h))
cv2.imshow("2. Rotated 120 Degrees", rotated_img)
```



The output size is kept at 640x480, so the corners rotate out of the frame and the empty areas are filled with black.

5.3 Shearing the image along the X axis

```
# 3. Perform Shear operations over the image (X-axis shear)
shear_factor= 0.5
M_shear= np.float32([[1, shear_factor, 0], [0, 1, 0]])
sheared_img= cv2.warpAffine(image, M_shear, (int(w + h * shear_factor), h))
cv2.imshow("3. Sheared Image", sheared_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Each row is shifted in proportion to its position, so the output canvas is widened to 880 pixels ($w + h \times 0.5$).

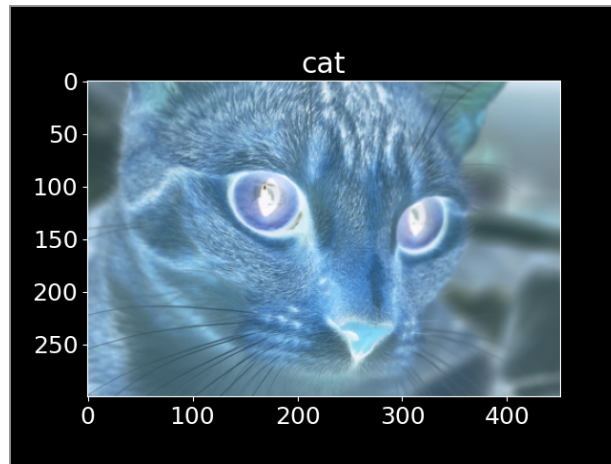
Task 6: Intensity Transformations (Point Processing)

- **Description:** Applied three intensity transformations: the image negative, the log transformation, and power-law (gamma) correction with gamma = 2.0, implemented with a look-up table and cv2.LUT.

- **Code & Output:**

6.1 Negative image

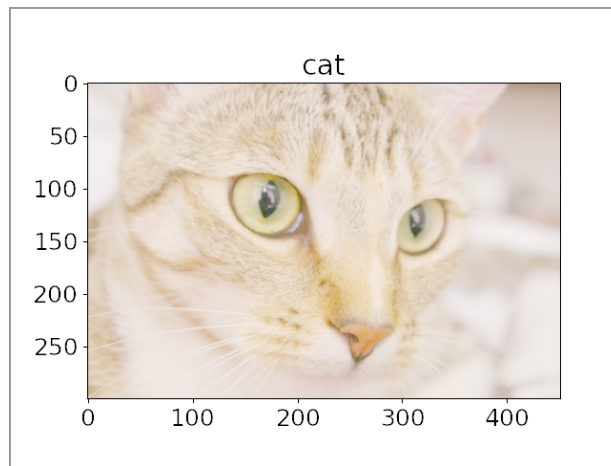
```
# 1. Create a Negative Image  
negative_image= 255 - image  
cv2.imshow("1. Negative Image", negative_image)
```



255 - image inverts every channel, so the white margins become black and the colors are inverted.

6.2 Log transformation

```
# 2. Log Transformation
image_float= np.float32(image)
c= 255 / np.log(1 + np.max(image_float))
log_image= c * (np.log(image_float + 1))
log_image= np.array(log_image, dtype= np.uint8)
cv2.imshow("2. Log Transformation", log_image)
```



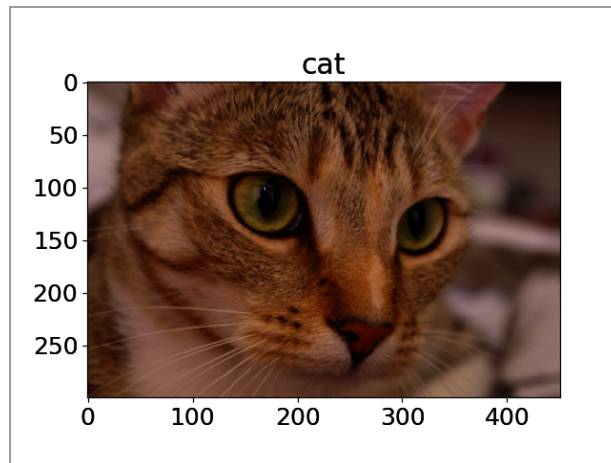
$c \cdot \log(1 + r)$ with $c = 255 / \log(1 + \max) \approx 46$ expands dark values and compresses bright ones, so the image becomes brighter and washed out.

6.3 Power-law (gamma) correction

```
# 3. Power Law (Gamma Correction)
gamma= 2.0
lookup_table= np.array([((i / 255.0) ** gamma) * 255 for i in np.arange(0,
256)]).astype("uint8")
power_low_img= cv2.LUT(image, lookup_table)
cv2.imshow("3. Power Law (Gamma= 2.0)", power_low_img)

# Wait for key press and close windows
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.waitKey(1)
```



With $\gamma = 2.0$ (greater than 1) the table maps each intensity i to $(i / 255)^2 \times 255$, which darkens the image and expands the bright regions.

3. What I Learned

- **Basic image I/O and display:** cv2.imread loads images in the default BGR format or in grayscale (cv2.IMREAD_GRAYSCALE), cv2.imshow renders them, cv2.waitKey pauses until a key is pressed, cv2.destroyAllWindows closes the windows, and cv2.imwrite saves array data back to disk.
- **Color space conversions:** cv2.cvtColor converts BGR to grayscale or YUV, and the YUV channels (Y: brightness, U: blue projection, V: red projection) can be isolated with NumPy slicing, `img[:, :, channel_index]`.
- **Geometric transformations:** cv2.resize scales an image with an interpolation method such as cv2.INTER_CUBIC, cv2.getRotationMatrix2D with cv2.warpAffine rotates it

around its center, and a custom 2x3 matrix such as `[[1, shear_x, 0], [0, 1, 0]]` with `cv2.warpAffine` shears it.

- **Intensity transformations:** Negatives subtract the pixel values from the maximum value ($255 - \text{img}$). The log transformation $c \cdot \log(1 + r)$ expands dark pixels and compresses bright ones, and needs float arrays to avoid overflow. Power-law correction $c \cdot r^{\text{gamma}}$ expands bright regions for gamma greater than 1 and dark regions for gamma less than 1, and is done efficiently with `cv2.LUT`.
- **Troubleshooting:** If image windows freeze in an IDE or notebook, close them with a key press instead of the red X, and follow `cv2.destroyAllWindows()` with a tiny delay such as `cv2.waitKey(1)`.